
Executable Construction Records for Guided Quantum Stabilizer Code Search

Yusheng Zhao
MBZUAI
yusheng.zhao@mbzuai.ac.ae

Abstract

A catalog can preserve how a stabilizer code was built without making that construction executable by a learning or search algorithm. We convert such descriptions into records pairing typed operations with stabilizer matrices and compare six random and guided methods across two representations on targets absent from training. At test time, methods receive only (n, k, L) and must produce an explicit construction and matrix, while known reference constructions remain hidden. In the original six-method comparison, all three guided methods produced more Pass outcomes than their matched random controls. MCTS and the neural policy achieved the two highest Pass rates, 11.48% and 10.61%, at about one-sixth of the aggregate runtime of genetic search. Under matched budgets, increasing maximum search depth from one to four produced 5.40% more Pass outcomes for the neural policy in aggregate, with an improvement in every seed. This method-specific result provides a promising signal that the neural policy may benefit from deeper exploration. Together, these results establish executable records as a practical testbed for learning-guided construction beyond the observed training targets.

1 Introduction

Qubit stabilizer codes [16] are a central framework for quantum error correction. A code $[[n, k, d]]_2$ encodes k logical qubits in n physical qubits; its minimum distance d , the least weight of a nontrivial logical Pauli operator, determines its error-correction capability. CodeTables [17] tabulates distance bounds and constructions for qubit codes with $n, k \leq 256$. A cataloged lower bound records that such a code exists; establishing the distance of an independently generated stabilizer matrix requires a separate evaluation.

Algebraic constructions [4, 13] and automated methods [31, 38] operate over classical-code families, stabilizer matrices, or encoder circuits under representation-specific objectives. Existing benchmarks address different tasks: the QEC Challenge [36] evaluates submitted quantum low-density parity-check (qLDPC) codes, StabilizerBench [25] targets circuit synthesis, and ECCentric [32] measures system-level code performance. We study a different question: can cataloged construction records be made executable and reused on target parameters absent from the database available to the search methods, and does deeper guided exploration improve the resulting verified outcomes under matched budgets?

We construct a benchmark for learning and search over executable stabilizer-code constructions. Its dataset contains 18,141 records, each pairing a typed construction sequence with its stabilizer matrix and cataloged distance interval. A component-based split yields 14,512 training records and 3,629 unique test targets, with no train-test overlap in (n, k) or (n, k, L) . At test time, methods receive (n, k, L) while the corresponding construction record and matrix are hidden. The benchmark provides six baseline methods across quasi-twisted (QT) and construction-sequence representations. A common evaluator validates the stabilizer structure and computes distance bounds using exhaustive

enumeration, the MacWilliams identities, and Brouwer–Zimmermann search. The metrics separate matching the requested (n, k) from establishing distance evidence and report outcome rates and runtime.

We evaluate six representative construction methods spanning randomized search, evolutionary optimization, tree search, and learned proposal models. In a paired five-seed depth screen, increasing maximum depth from one to four produced 5.40% more Pass outcomes for the neural policy in aggregate, with an improvement in every seed, suggesting that it may benefit from deeper exploration. In the original six-method comparison, all three guided methods produced more Pass outcomes than their matched random controls. These results establish executable construction records as a practical foundation for learning-guided stabilizer-code construction beyond the observed training targets. Anonymous code is available at this *anonymous repository*.

2 Background

We first define stabilizer codes and distance, then describe the two construction representations and their relation to prior work.

2.1 Stabilizer representation and distance

An n -qubit Pauli operator is represented, up to phase, by $v = (x \mid z) \in \mathbb{F}_2^{2n}$. For $v = (x \mid z)$ and $w = (x' \mid z')$, the symplectic inner product is $\langle v, w \rangle_s = xz'^T + zx'^T$. The operators commute exactly when this value is zero. We write every stabilizer matrix as $H = [H_X \mid H_Z] \in \mathbb{F}_2^{r \times 2n}$, where H_X and H_Z contain the X and Z components of the same r generators. Its rows commute when

$$H_X H_Z^T + H_Z H_X^T = 0. \quad (1)$$

The resulting code encodes $k = n - \text{rank}(H)$ logical qubits. Redundant rows do not change the code, and different generator matrices can represent the same row space. We therefore identify and deduplicate candidate matrices by their stabilizer row space.

Let h_i be row i of H . A Pauli error is undetectable and nontrivial when it commutes with every stabilizer generator but is not generated by them. For $k > 0$, define

$$\begin{aligned} \mathcal{U}(H) &= \{e \in \mathbb{F}_2^{2n} : \langle h_i, e \rangle_s = 0 \forall i, e \notin \text{rowspan}(H)\}, \\ d(H) &= \min_{e \in \mathcal{U}(H)} \text{wt}_P(e). \end{aligned} \quad (2)$$

Here wt_P counts qubits on which the X/Z pair is nonzero. For a stabilizer state with $k = 0$, $\mathcal{U}(H)$ is empty. We instead report the minimum weight of a nonidentity stabilizer.

2.2 Code-construction representations

Several construction operations begin with binary or additive $\mathbb{F}_4$ codes. Rank, duality, self-orthogonality, and the hull $C \cap C^\perp$ determine whether a classical source yields a stabilizer. Under the standard $\mathbb{F}_4$ correspondence [4], trace-Hermitian and binary symplectic orthogonality are equivalent. A Calderbank–Shor–Steane (CSS) matrix [3, 30] can be ordered as

$$H = \left[\begin{array}{c|c} H_X^{\text{CSS}} & 0 \\ \hline 0 & H_Z^{\text{CSS}} \end{array} \right], \quad H_X^{\text{CSS}} (H_Z^{\text{CSS}})^T = 0.$$

The algebraic representation searches quasi-twisted (QT) codes whose block generators are polynomials modulo $x^m - \lambda$. The block size and twist determine the quotient ring; coefficients select a code, and duality and hull conditions determine whether it yields a stabilizer directly or through Construction X [13].

The second representation uses sequences of typed construction operations. Each operation specifies its operands and parameters and is validated before execution. The operations include source initialization, changes in dimension and coordinates, CSS construction, and code composition. Both representations produce a stabilizer matrix for the same evaluation procedure.

2.3 Related work

Work on automated QEC construction [5, 31, 23, 38, 18, 24, 35, 22, 33, 15, 26] searches encoder circuits, stabilizer matrices, and parameterized code families. Recent systems [10, 11, 21, 40, 28, 29] extend this scope to structured programs and joint code–circuit–decoder objectives. These systems optimize within a chosen representation, family, or deployment objective. We instead test whether recorded construction operations can be reused for held-out (n, k, L) targets under a common matrix-based evaluation.

Catalogs and construction benchmarks serve a different role. CodeTables [17] and the Error Correction Zoo [1] collect known parameters and constructions. Other benchmarks [7, 32, 25] compare code families under coding or system metrics or synthesize circuits for specified codes. The contemporaneous QEC Challenge [36] accepts CSS qLDPC check pairs, recomputes structural properties, and uses explicit logical operators as upper-bound witnesses. Our benchmark starts from targets with held-out reference constructions, accepts general non-CSS $[H_X \mid H_Z]$ matrices, records the construction sequence for each candidate, and keeps Pass, Fail, Unresolved, and No match distinct under a fixed computational budget.

Symbolic and formal systems [39, 14, 20, 8] verify QEC programs and fault-tolerant operations. Given a code, QDistRnd [27] estimates upper bounds; Brouwer–Zimmermann search [19] and SAT/MaxSAT solvers [9] perform exact-distance searches or threshold decisions; and machine-checked certificates [12] formally verify results. The contemporaneous codeDistance package [37] unifies exact and heuristic routines across code and circuit inputs. We use such routines as distance-computation backends in the common evaluator for executable construction records.

3 Benchmark design

We define the held-out task, the six construction methods, and the outcome and runtime metrics used for comparison.

3.1 Data and task

Each CodeTables record [17] is a four-tuple $s_i = (\tau_i, H_i, L_i, U_i)$, where $\tau_i = (a_{i1}, \dots, a_{iT_i})$ is a recorded construction sequence, $H_i = [H_{X,i} \mid H_{Z,i}]$ is its resulting stabilizer matrix, and $[L_i, U_i]$ is the catalog distance interval. The pair (n_i, k_i) follows from H_i . Each record defines one target $x_i = (n_i, k_i, L_i)$.

We map the recorded actions to nine operations for initialization, transformation, and composition. Eight are available to the search methods; ImportWitness initializes a sequence from a stored code. Their definitions and corpus frequencies appear in Appendix A.

For target $x = (n, k, L)$, method m must produce a candidate construction $\pi \in \Pi_m(x)$ whose deterministic output $H_m(\pi)$ satisfies

$$H_m(\pi) \in \mathbb{F}_2^{r \times 2n} \text{ is a valid stabilizer,} \quad k(H_m(\pi)) = k, \quad d(H_m(\pi)) \geq L. \quad (3)$$

Here π specifies either a QT genome and its deterministic decoding procedure or a valid sequence of construction operations. Each method returns an explicit construction and stabilizer matrix rather than predicting existence. A candidate is classified as Pass only if the terminal evaluator establishes $d(H) \geq L$. U_i is diagnostic only, and $k = 0$ targets use the minimum stabilizer weight defined in Section 2.1.

3.2 Baseline methods

Figure 1 groups the six methods by construction representation.

Algebraic search over quasi-twisted codes. The algebraic methods search parameterizations of $\mathbb{F}_4$ Hermitian and $\mathbb{F}_2$ symplectic QT codes. A deterministic decoder applies rank, orthogonality, hull, and ConstructionX [13] checks before returning a stabilizer with the requested (n, k) . Random QT search evaluates valid decoded candidates directly. Matched random QT search samples the same genomes independently, whereas genetic QT search selects, crosses, and mutates them using a proxy

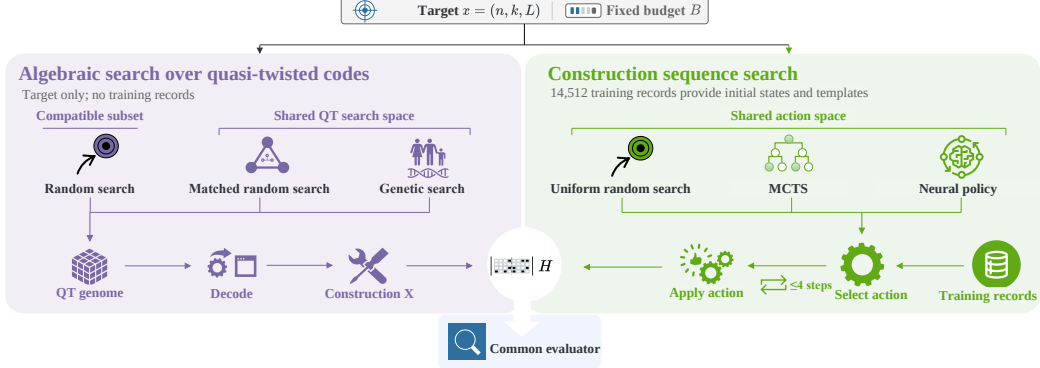


Figure 1: **Six construction baselines.** The methods search either algebraic representations or sequences of construction operations. Paired comparisons use methods with the same representation.

score based on low-weight logical operators. The score ranks candidates, while the terminal evaluator determines *Pass*; candidates with the same stabilizer row space are evaluated once.

Search over construction sequences. These methods share training-set initial states, recorded action templates, and a transition function. Each action specifies its operands and parameters; the resulting state must be valid, canonical, and consistent with its parent.

Uniform random action selection samples first among applicable action types and then among their candidate operations. MCTS searches the same states and actions with upper-confidence selection and progressive widening. Intermediate distance bounds prioritize candidates, while the terminal evaluator determines *Pass*. A frozen neural policy trained on recorded actions ranks action types and candidate operations without online updates.

3.3 Evaluation metrics

The terminal evaluator reports $[\ell(H), u(H)] \ni d(H)$ for one deterministically selected candidate in each target–method pair. Appendix Section B defines candidate selection and the secondary evaluation diagnostics.

Outcome rates. For target i and method m , Y_{im} is pass, fail, unresolved, or no match. For outcome y , its rate is

$$r_{m,y} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[Y_{im} = y]. \quad (4)$$

Pass rate is primary. Fail means $u(H) < L$, Unresolved means $\ell(H) < L \leq u(H)$, and No match means that no valid submitted candidate has the requested (n, k) . For paired methods, the Pass-rate difference is $(n_{10} - n_{01})/N$, where n_{10} and n_{01} count targets classified as Pass by only the first or second method, respectively.

Runtime. We report process CPU time and aggregate per-target wall-clock time, defined as the sum of target-level measurements including evaluator calls.

4 Common terminal evaluation

Each method submits $H = [H_X \mid H_Z]$ to the common terminal evaluator in Figure 2, which uses neither the hidden construction nor its catalog distance interval. Algorithm 1 gives the procedure for $k > 0$.

Structural verification. The evaluator canonicalizes the binary rows over $\mathbb{F}_2$, tests Eq. (1), and recovers $\hat{n} = \text{ncol}(\bar{H})/2$ and $\hat{k} = \hat{n} - \text{rank}(\bar{H})$. It rejects malformed or noncommuting matrices

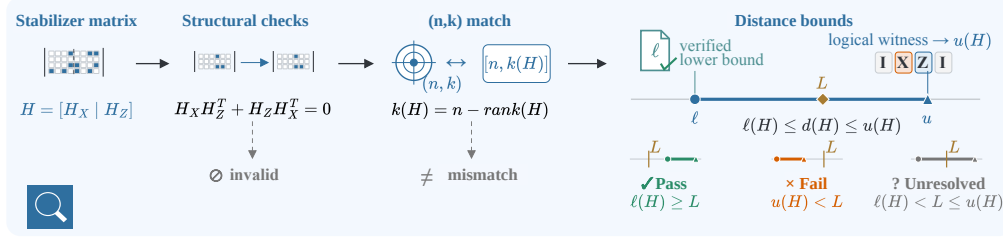


Figure 2: **Common terminal evaluator.** Every submitted stabilizer matrix undergoes the same checks of structural validity, code parameters, and distance bounds.

Algorithm 1 Candidate evaluation using distance bounds

```

1: Input target  $x = (n, k, L)$  with  $k > 0$ , candidate  $H$ , and remaining budget  $B$ 
2:  $(s, \bar{H}, \hat{n}, \hat{k}) \leftarrow \text{CANONICALIZECANDIDATE}(H)$  ▷ Algorithm 2
3: if  $s = \text{invalid}$  then
4:   return invalid
5: else if  $(\hat{n}, \hat{k}) \neq (n, k)$  then
6:   return mismatched
7:  $(\ell, u) \leftarrow \text{COMPUTEDISTANCEBOUNDS}(\bar{H}, B)$  ▷ Algorithm 3
8: if  $\ell \geq L$  then
9:   return Pass
10: else if  $u < L$  then
11:   return Fail
12: else
13:   return Unresolved

```

and candidates with incorrect (n, k) . Canonical row spaces make this decision invariant to redundant generators and basis changes.

Distance-bound computation. For a structurally valid candidate with the requested (n, k) , the evaluator maintains

$$\ell(H) \leq d(H) \leq u(H). \quad (5)$$

Exhaustive enumeration by Pauli weight raises ℓ after all operators of a given weight have been excluded and determines the distance exactly when it finds a logical operator. If its budget is exhausted, small-rank candidates use the MacWilliams transform for additive codes [4]; larger candidates use Brouwer–Zimmermann search [19] in Qubitserf [6]. In the MacWilliams calculation, A_j and B_j count weight- j stabilizers and normalizer elements, respectively; the first $j \geq 1$ with $B_j - A_j > 0$ is the logical distance.

An exact result is accepted only if the computation terminates successfully and d_* is an integer satisfying $1 \leq \ell \leq d_* \leq u \leq n$ for the canonical input. Otherwise, the bounds remain unchanged. The exact-distance routine is selected from candidate properties, not from the proposal method.

Appendix B.1 gives the subroutines called by Algorithm 1.

Evidence handling. The lower bound increases only after exhaustive enumeration of a weight class, and every logical witness is rechecked against Eq. (2). A completed Qubitserf result with its proven flag supplies an exact value. An incomplete or inconsistent result leaves the distance bounds unchanged. These rules define the common evaluation protocol used throughout the benchmark.

5 Experiments

We first specify the data split and budgets, then examine maximum search depth, guided-versus-random comparisons, outcome rates, and calibration using hidden reference codes.

Table 1: **Maximum-depth terminal outcomes.** Cell counts aggregate 3,581 targets over five evaluation seeds ($N = 17,905$ target-seed cells per method); Pass rates are in parentheses.

Method	$D_{\max}$	Cells	Match	Pass	Fail	Unresolved	No match
MCTS over construction actions	4	17,905	17,905	1,985 (11.09%)	1,681	14,239	0
selection Uniform action sampling	4	17,905	17,905	1,901 (10.62%)	1,534	14,470	0
Neural policy for action	4	17,905	17,905	2,070 (11.56%)	2,936	12,899	0

5.1 Experimental setup

We linked records with the same construction description or a shared input or output code into 1,204 components. Keeping components intact produced 14,512 training records and 3,629 test records in 213 test-set components. All test (n, k, L) targets are unique, with no overlap between the training and test sets in either (n, k) or (n, k, L) .

The training set supplied initial states, construction templates, and action labels to methods that searched construction sequences; the algebraic methods used no training records. At test time, every method received only $x = (n, k, L)$; τ_i , H_i , and U_i were not used to rank candidates or evaluate the distance threshold. Neural training and complete method configurations appear in Appendix B.

In the original six-method comparison, each target-method run allowed 64 proposals or actions and four evaluator calls sharing 10^6 Pauli tests; MCTS used 256 simulations and genetic search used a population of eight. For the supplementary depth experiments, we separately froze search and verification budgets. Complete settings and contract checks appear in Appendix B.

Results were complete for 3,627 of 3,629 test targets, comprising 3,581 stabilizer codes with $k > 0$ and 46 stabilizer states with $k = 0$. Outcome rates use the common denominator $N = 3,581$ for targets with $k > 0$ because stabilizer states use a different evaluation criterion.

5.2 Experimental results

Maximum search depth. For each sequence method, we paired $D_{\max} = 1$ and 4 on the same 3,581 targets and five seeds, with matched search and verification budgets. The $D_{\max} = 4$ minus $D_{\max} = 1$ Pass-rate differences (95% two-way cluster-bootstrap intervals) were -0.011 percentage points for uniform action sampling ($[-0.080, 0.048]$), $+0.034$ for MCTS ($[-0.139, 0.168]$), and $+0.594$ for the neural policy ($[0.226, 1.624]$). Only the neural interval excluded zero, and its difference was positive for all five seeds. Depth-4-only Pass cells with observed sequence length at least two numbered 2, 40, and 3, respectively. The neural result therefore passes the prespecified screen for a full depth curve.

Table 1 reports the terminal outcome composition at $D_{\max} = 4$.

Guided versus random search. We compared each guided method with its random baseline on the same 3,581 test targets with $k > 0$. Each pair used the same construction representation and number of evaluator calls. Table 2 compares identical targets within each representation. Genetic QT search, MCTS, and the neural policy classified 19, 60, and 29 additional targets as Pass relative to their random baselines. All three Pass-rate differences were positive.

Table 2: **Paired comparison of Pass outcomes.**

Method	Guided only	Baseline only	Δ (pp)
Genetic QT	27	8	+0.531
MCTS	83	23	+1.676
Neural policy	59	30	+0.810

Sensitivity to the number of evaluator calls. Using the four recorded evaluator calls, we computed the cumulative Pass rate after each call. A target was counted at call j if any result through that call was Pass. Figure 3 shows that genetic QT search exceeded its baseline at every call and classified 131 versus 112 targets as Pass after four calls. MCTS exceeded its baseline after the first call and reached 411 versus 351, whereas the neural policy first exceeded its baseline at the fourth call, reaching 380 versus 351.

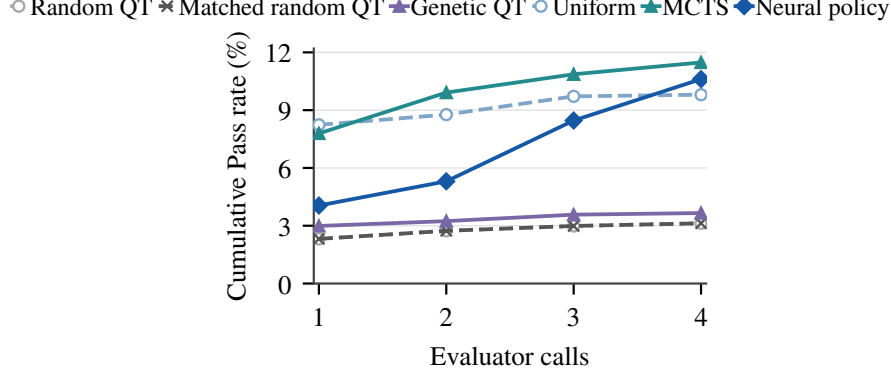


Figure 3: **Cumulative Pass rate by evaluator call.**

Outcome rates and runtime. Following the benchmark metrics in Section 3.3, we report the four exclusive outcomes on 3,581 targets with $k > 0$ and runtime on all 3,627 completed targets. Figure 4a shows that, among the original six methods, MCTS and the neural policy achieved the two highest Pass rates, 11.48% and 10.61%, while every construction-sequence method matched the requested (n, k) . Figure 4b shows aggregate per-target wall-clock times of 18.8 h for MCTS and 17.2 h for the neural policy, compared with 110.0 h for genetic search; random QT search was fastest overall. Within this original comparison, MCTS and the neural policy therefore combined the highest Pass rates with aggregate times about one-sixth of that for genetic search.

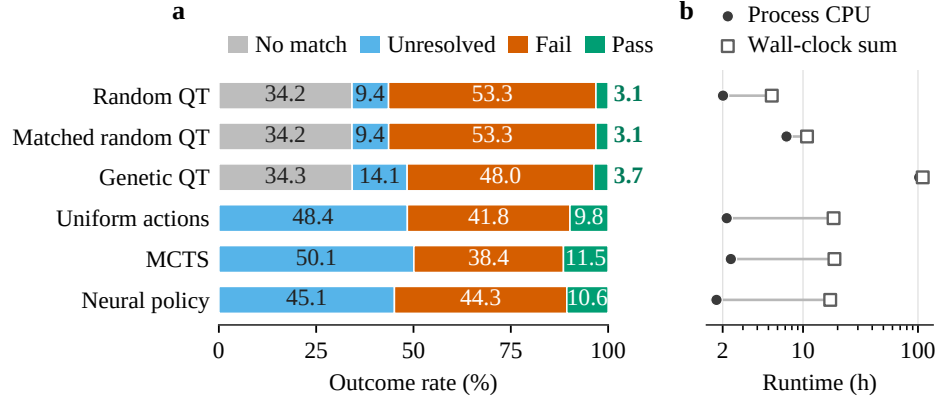


Figure 4: **Outcome rates and runtime.** Panel a reports Pass, Fail, Unresolved, and No match rates over 3,581 completed test targets with $k > 0$. Panel b reports process CPU time and the sum of per-target wall-clock times over all 3,627 completed test targets. Process CPU time excludes native child processes.

Outcomes by distance threshold. We stratified the same 3,581 targets with $k > 0$ by required distance L and retained the method-specific Pass and Unresolved rates. Figure 5 shows that Pass rates were at most 4.8% for $L \geq 4$. Only MCTS classified any $L = 7-8$ target as Pass (6/476), and no method classified a target as Pass at $L \geq 9$. In these two high- L groups, 86.5%–95.8% of outcomes from methods that search over construction sequences were Unresolved. We next applied the same evaluation budget to the hidden reference codes.

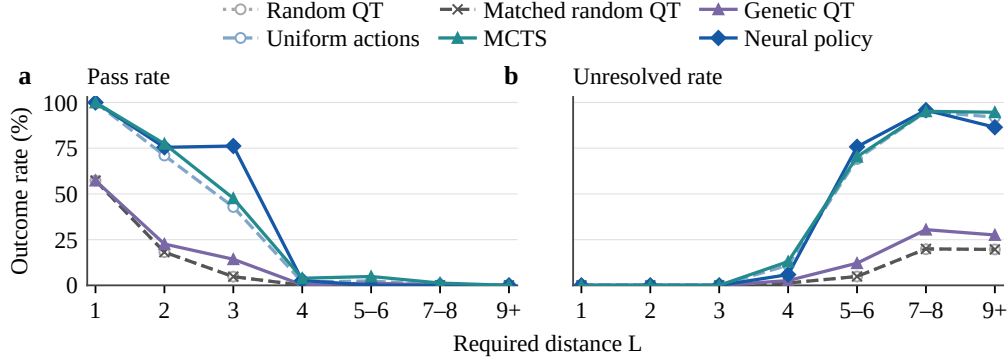


Figure 5: **Outcome rates by required distance.** Curves report percentages of test targets with $k > 0$; group sizes from $L = 1$ to $L \geq 9$ are 89, 314, 21, 1,416, 165, 476, and 1,100.

Calibration using hidden reference codes. We evaluated all 3,629 hidden reference matrices under the same computational budget; the search methods never accessed them. Construction sequences and catalog bounds were excluded. Table 3 shows that all 404 reference codes with $k > 0$ and $L = 1-2$ were classified as Pass, compared with 19 of 476 at $L = 7-8$ and 2 of 1,100 at $L \geq 9$; none was classified as Fail. High- L reference outcomes therefore measure the evaluator’s distance-resolution coverage under the benchmark budget.

Table 3: **Outcomes for hidden reference codes by required distance.**

Reference codes	N	Pass	Fail	Unresolved
$k > 0, L = 1-2$	404	404	0	0
$k > 0, L = 3-4$	1,437	1,033	0	404
$k > 0, L = 5-6$	165	45	0	120
$k > 0, L = 7-8$	476	19	0	457
$k > 0, L \geq 9$	1,100	2	0	1,098
Stabilizer states	47	5	0	42

6 Conclusion

We converted cataloged stabilizer-code constructions into executable records and assembled them into a benchmark for learning and search on targets absent from training. In a budget-matched five-seed ablation, increasing maximum search depth from one to four produced 5.40% more Pass outcomes for the neural policy in aggregate, with an improvement in every seed, while uniform sampling and MCTS showed no stable gain. This method-specific gain provides a promising signal that the neural policy may benefit from deeper exploration. Across both representations in the original six-method comparison, guided methods also produced more Pass outcomes than their matched random controls. Together, the dataset, evaluator, and baselines form a practical benchmark for learning-guided stabilizer-code construction beyond the observed training targets.

Discussion & Limitations

Parameter matching, candidate selection, and distance resolution are distinct parts of the construction task. The positive guided-versus-random comparisons indicate that genetic search, MCTS, and the neural policy prioritize candidates more effectively than random search within their respective representations.

The original six-method comparisons use one evaluation seed, and internal computation is not matched across representations. Runtime accounting excludes offline neural training and process CPU used by native child processes. The advantage of the neural policy in the original comparison appears at the fourth evaluator call, so those paired outcomes also depend on the terminal-call budget. Absolute comparisons across representations and recorded runtimes remain descriptive.

The evaluator combines existing distance routines and treats completed Qubitserf results as trusted exact outputs; it is not a formal proof system. It resolved few high- L thresholds, and many hidden reference matrices remained Unresolved under the same evaluation budget. Outcomes in this regime reflect both candidate construction and the evaluator’s ability to establish the required distance evidence. The calibration measures evaluator resolution on recorded references rather than an upper bound on alternative constructions.

Finally, every test target has a cataloged reference construction that is hidden from the search methods. The benchmark therefore evaluates held-out targets with known reference constructions rather than targets with no recorded construction. Future experiments should match computational budgets across representations to separate search guidance from computational effort. Targets without cataloged constructions will test extrapolation beyond known records.

References

- [1] Victor V. Albert and Philippe Faist. The error correction zoo, June 2026. URL <https://errorcorrectionzoo.org/>.
- [2] Reinald Baart, Tom Boothby, Jasper Cramwinckel, Joe Fields, David Joyner, Robert Miller, Eric Minkes, Erik Roijackers, Lea Ruscio, and Cen Tjhai. GUAVA: A GAP package for computing with error-correcting codes, version 3.21, February 2026. URL <https://gap-packages.github.io/guava/>.
- [3] A. Robert Calderbank and Peter W. Shor. Good quantum error-correcting codes exist. *Physical Review A*, 54(2):1098–1105, August 1996.
- [4] A. Robert Calderbank, Eric M. Rains, Peter W. Shor, and Neil J. A. Sloane. Quantum error correction via codes over $GF(4)$. *IEEE Transactions on Information Theory*, 44(4):1369–1387, July 1998.
- [5] Chenfeng Cao, Chao Zhang, Zipeng Wu, Markus Grassl, and Bei Zeng. Quantum variational learning for quantum error-correcting codes. *Quantum*, 6:828, October 2022.
- [6] Serban Cercelescu. Qubitserf: Quantum coding theory library, June 2026. URL <https://github.com/Quantinuum/qubitserf>.
- [7] Avimita Chatterjee and Swaroop Ghosh. Magic mirror on the wall, how to benchmark quantum error correction codes, overall? In *Proceedings of the 2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*, pages 356–367, September 2024.
- [8] Kean Chen, Yuhao Liu, Wang Fang, Jennifer Paykin, Xin-Chuan Wu, Albert Schmitz, Steve Zdancewic, and Gushu Li. Verifying fault-tolerance of quantum error correction codes. In *Proceedings of the 37th International Conference on Computer Aided Verification (CAV)*, pages 3–27, July 2025.
- [9] Yu-Fang Chen, Seyed Mohammad Reza Jafari, and Ching-Yi Lai. SAT, MaxSAT, and SMT for QLDPC distance computation: A large-scale empirical study. *arXiv preprint arXiv:2606.12445*, May 2026.

- [10] Yihua Chengyu, Richard Meister, Conor Carty, Sheng-Ku Lin, and Roberto Bondesan. Bayesian optimization for quantum error-correcting code discovery. *arXiv preprint arXiv:2601.18562*, January 2026.
- [11] Juan Cruz-Benito, Andrew W. Cross, David Kremer, and Ismael Faro. Evolutionary discovery of bivariate bicycle codes with LLM-guided search. *arXiv preprint arXiv:2606.02418*, June 2026.
- [12] Mattias Ehatamm, Yi Lee, Xiaodi Wu, and Runzhou Tao. End-to-end formalization of quantum error correction. *arXiv preprint arXiv:2605.16523*, May 2026.
- [13] Martianus Frederic Ezerman, Markus Grassl, San Ling, Ferruh Özbudak, and Buket Özkaya. Characterization of nearly self-orthogonal quasi-twisted codes and related quantum codes. *IEEE Transactions on Information Theory*, 71(1):499–517, January 2025.
- [14] Wang Fang and Mingsheng Ying. Symbolic execution for quantum error correction programs. *Proceedings of the ACM on Programming Languages*, 8(PLDI):1040–1065, June 2024.
- [15] Bruno C. A. Freire, Nicolas Delfosse, and Anthony Leverrier. Optimizing hypergraph product codes with random walks, simulated annealing and reinforcement learning. In *Proceedings of the 2025 IEEE International Symposium on Information Theory*, pages 1–6, 2025. doi: 10.1109/ISIT63088.2025.11195424.
- [16] Daniel Eric Gottesman. Stabilizer codes and quantum error correction (ph.d. thesis, california institute of technology), May 1997. URL <https://thesis.library.caltech.edu/2900/>.
- [17] Markus Grassl. Bounds on the minimum distance of quantum codes, May 2024. URL <https://www.codetables.de/QECC/index.html>.
- [18] Austin Yubo He and Zi-Wen Liu. Discovering highly efficient low-weight quantum error-correcting codes with reinforcement learning. *arXiv preprint arXiv:2502.14372*, February 2025.
- [19] Fernando Hernando, Gregorio Quintana-Ortí, and Markus Grassl. Fast algorithms and implementations for computing the minimum distance of quantum codes. *ACM Transactions on Quantum Computing*, 7(2):1–19, June 2026.
- [20] Qifan Huang, Li Zhou, Wang Fang, Mengyu Zhao, and Mingsheng Ying. Efficient formal verification of quantum error correcting programs. *Proceedings of the ACM on Programming Languages*, 9(PLDI):1068–1093, June 2025.
- [21] Zidu Liu and Florian Marquardt. Large-language-model discovery of quantum LDPC codes through structured concept evolution. *arXiv preprint arXiv:2606.24808*, June 2026.
- [22] Caroline Maunon, Terry Farrelly, and Thomas M. Stace. Optimization of tensor network codes with reinforcement learning. *New Journal of Physics*, 26:023024, 2024. doi: 10.1088/1367-2630/ad23a6.
- [23] Jan Olle, Remmy Zen, Matteo Puviani, and Florian Marquardt. Simultaneous discovery of quantum error correction codes and encoders with a noise-aware reinforcement learning agent. *npj Quantum Information*, 10(1):126, December 2024.
- [24] Jan Olle, Oleg M. Yevtushenko, and Florian Marquardt. Scaling the automated discovery of quantum circuits via reinforcement learning with gadgets. *arXiv preprint arXiv:2503.11638*, March 2025.
- [25] Andres Paz, Christian Tarta, Cordelia Yuqiao Li, Mayee Sun, Sarju Patel, and Sylvie Lausier. StabilizerBench: A benchmark for AI-assisted quantum error correction circuit synthesis. *arXiv preprint arXiv:2604.21287*, April 2026.
- [26] Michael A. Perlin, Matthew Steinberg, and Ben Criger. Quantum codes from classical annealing. *arXiv preprint arXiv:2607.19574*, July 2026.

- [27] Leonid P. Pryadko, Vadim A. Shabashov, and Valerii K. Kozin. QDistRnd: A GAP package for computing the distance of quantum error-correcting codes. *Journal of Open Source Software*, 7(71):4120, March 2022.
- [28] Dongheng Qian and Tianyi Li. Multi-agent discovery of practical quantum LDPC codes. *arXiv preprint arXiv:2608.08996*, August 2026.
- [29] Omer S. Sella, Robert Pinsler, and Thomas Heinis. Designing quantum error correcting codes to fit decoders via reinforcement learning. *arXiv preprint arXiv:2608.15754*, August 2026.
- [30] Andrew M. Steane. Multiple-particle interference and quantum error correction. *Proceedings of the Royal Society of London. Series A: Mathematical, Physical and Engineering Sciences*, 452(1954):2551–2577, November 1996.
- [31] Vincent Paul Su, ChunJun Cao, Hong-Ye Hu, Yariv Yanay, Charles Tahan, and Brian Swingle. Discovery of optimal quantum codes via reinforcement learning. *Physical Review Applied*, 23(3):034048, March 2025.
- [32] Aleksandra Świerkowska, Jannik Pflieger, Emmanouil Giortamis, and Pramod Bhatotia. ECCentric: An empirical analysis of quantum error correction codes. *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, 10(2):1–33, May 2026. doi: 10.1145/3805635.
- [33] Daniel Tandeitnik and Thiago Guerreiro. Evolving quantum circuits. *Quantum Information Processing*, 23:109, 2024. doi: 10.1007/s11128-024-04317-w.
- [34] The GAP Group. GAP: Groups, algorithms, and programming, version 4.14.0, December 2024. URL <https://www.gap-system.org/>.
- [35] Maxime Tremblay, Guillaume Duclos-Cianci, and Stefanos Kourtis. Finite-rate sparse quantum codes aplenty. *Quantum*, 7:985, 2023. doi: 10.22331/q-2023-04-20-985.
- [36] Unitary Foundation. QEC Challenge, June 2026. URL <https://unitaryfoundation.github.io/qldpc-challenge/>.
- [37] Mark Webster, Abraham Jacob, and Oscar Higgott. Distance-finding algorithms for quantum codes and circuits. *arXiv preprint arXiv:2603.22532*, March 2026.
- [38] Mark A. Webster and Dan E. Browne. Engineering quantum error correction codes using evolutionary algorithms. *IEEE Transactions on Quantum Engineering*, 6:1–14, February 2025.
- [39] Anbang Wu, Gushu Li, Hezi Zhang, Gian Giacomo Guerreschi, Yuan Xie, and Yufei Ding. QECV: Quantum error correction verification. *arXiv preprint arXiv:2111.13728*, November 2021.
- [40] Ge Yan, Shanchuan Li, Pengyue Ma, Qixin Zhang, Pingchuan Ma, Jianping Wang, Min-Hsiu Hsieh, and Yuxuan Du. OmniQEC: Discovering practical quantum error-correcting codes by an AI scientist. *arXiv preprint arXiv:2607.25865*, July 2026.

A Training corpus details

We first summarize the normalized action vocabulary, then give representative construction records and their stabilizer matrices.

A.1 Construction sequences and action counts

Of the 14,512 training records, 13,566 contain multiple steps, yielding 39,212 normalized actions. These include 2,342 `ImportWitness` actions supplied by the environment and 36,870 actions available to the search methods. Tables 4 and 5 report their frequencies and the mapping implemented by the parser.

Table 4: Counts of normalized construction actions in the training corpus.

Action	Count	Action	Count
Source	10,990	ImportWitness	2,342
ExtendCode	8,474	Subcode	8,543
CoordinateMinor	8,167	CSSLift	35
DirectSum	66	Concatenate	98
ConstructionX	497	Total	39,212

Table 5: **Mapping from recorded labels to action types.** The parser maps 27 recorded labels, shown in CamelCase, to nine action types. `ImportWitness` is supplied by the environment; the other eight are available to the search methods.

Action type	Recorded labels
<i>Initialization and imported inputs</i>	
ImportWitness	StoredGenerator
Source	UniverseCode, DistanceTwoSource, CyclicPolynomialSource, AdditiveCyclicPolynomialSource, ConstacyclicPolynomialSource, QuasiCyclicSource, AdditiveQuasiCyclicSource, LinearGF4QuasiCyclicSource, QuasiTwistedSource, BCHSource, ExtendedBCHSource, QuantumTwistedSource, and ExtendedQuantumTwistedSource
<i>Manipulation and composition</i>	
ConstructionX	QuantumConstructionX, ConstructionXParameterOnly, and ConstructionXXParameterOnly
Subcode	Subcode
ExtendCode	ExtendCode and StandardLengthening
CoordinateMinor	Puncture, Shortening, StabilizerShortening, and CSSShortenPuncture
CSSLift	CSSFromClassical
DirectSum	DirectSum
Concatenate	Concatenation

A.2 Illustrative construction records

The six examples cover all eight action types available to the search methods. Examples 1–3 show complete matrices, and Examples 4–6 show matrix excerpts. Catalog distance intervals are metadata and are not used to verify generated candidates.

Compact examples.

Training example 1. Source followed by Subcode

Record. `known:00010` produces $[[4, 1]]$ with catalog interval $[2, 2]$.

Recorded construction.

$$\emptyset \xrightarrow{\text{Source}} [[4, 2]] \xrightarrow{\text{Subcode}} [[4, 1]].$$

Here `Source` is the recorded distance-two presentation.

Complete final stabilizer matrix.

$$H = \left[\begin{array}{cccc|cccc} 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 \end{array} \right] \in \mathbb{F}_2^{3 \times 8}.$$

Training example 2. ImportWitness followed by ExtendCode

Record. known: 00021 produces $[[6, 1]]$ with catalog interval $[3, 3]$.

Recorded construction.

$$[[5, 1]]_{\text{ImportWitness}} \xrightarrow{\text{ExtendCode}(1)} [[6, 1]].$$

Complete final stabilizer matrix.

$$H = \left[\begin{array}{cccccc|cccc} 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \end{array} \right] \in \mathbb{F}_2^{5 \times 12}.$$

Training example 3. Subcode followed by ExtendCode

Record. known: 00030 produces $[[7, 3]]$ with catalog interval $[2, 2]$.

Recorded construction.

$$\emptyset \xrightarrow{\text{Source}} [[6, 4]] \xrightarrow{\text{Subcode}} [[6, 3]] \xrightarrow{\text{ExtendCode}(1)} [[7, 3]].$$

Complete final stabilizer matrix.

$$H = \left[\begin{array}{cccccc|cccccc} 1 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \end{array} \right] \in \mathbb{F}_2^{4 \times 14}.$$

Longer and compositional examples.

Training example 4. A five-step construction sequence

Record. known: 00578 produces $[[33, 18]]$ with catalog interval $[4, 6]$.

Recorded construction.

$$\begin{aligned} \emptyset &\xrightarrow{\text{Source}} [[40, 30]] \xrightarrow{\text{CoordinateMinor}} [[30, 20]] \xrightarrow{\text{Subcode}} [[30, 18]] \\ &\xrightarrow{\text{ExtendCode}(2)} [[32, 18]] \xrightarrow{\text{ExtendCode}(1)} [[33, 18]]. \end{aligned}$$

Here Source is a quasi-cyclic presentation and CoordinateMinor shortens ten recorded coordinates.

Excerpt of the final stabilizer matrix.

$$H = \left[\begin{array}{cccc|cccc} 10000011 \cdots 10001000 & 00000001 \cdots 11010000 \\ 00000001 \cdots 01010000 & 10000010 \cdots 10000000 \\ 01000000 \cdots 00011000 & 00000001 \cdots 00111000 \\ \vdots & \vdots \\ 00000000 \cdots 00000010 & 00000000 \cdots 00000000 \\ 00000000 \cdots 00000001 & 00000000 \cdots 00000000 \end{array} \right] \in \mathbb{F}_2^{15 \times 66}.$$

Training example 5. Concatenation followed by subcoding and puncturing

Record. known: 08564 produces $[[134, 2]]$ with catalog interval $[26, 45]$.

Recorded construction.

$$\begin{aligned} ([[27, 3]], [[5, 1]]) &\xrightarrow{\text{Concatenate}} [[135, 3]] \\ &\xrightarrow{\text{Subcode}} [[135, 2]] \xrightarrow{\text{CoordinateMinor}} [[134, 2]]. \end{aligned}$$

The final CoordinateMinor punctures coordinate 135.

Excerpt of the final stabilizer matrix.

$$H = \left[\begin{array}{cc|c} 10011000 \dots 00000001 & 00011000 \dots 00000011 & \\ 00011000 \dots 00000011 & 10000000 \dots 00000101 & \\ 01010000 \dots 00000011 & 00011000 \dots 00000101 & \\ \vdots & \vdots & \\ 00000000 \dots 00000011 & 00000000 \dots 00001110 & \\ 00000000 \dots 00000111 & 00000000 \dots 00000000 & \end{array} \right] \in \mathbb{F}_2^{132 \times 268}.$$

Training example 6. A branching construction with DirectSum

Record. known: 16481 produces the stabilizer state $[[240, 0]]$ with catalog interval $[20, 72]$.

Recorded construction graph.

$$\begin{aligned} \emptyset &\xrightarrow{\text{Source}} [127, 64] \xrightarrow{\text{ExtendCode}(1)} [128, 64] \xrightarrow{\text{CSSLift}} [[128, 0]], \\ \emptyset &\xrightarrow{\text{Source}} [111, 57] \xrightarrow{\text{ConstructionX}(e=3)} [[114, 0]] \xrightarrow{\text{CoordinateMinor}} [[112, 0]], \\ ([[112, 0]], [[128, 0]]) &\xrightarrow{\text{DirectSum}} [[240, 0]]. \end{aligned}$$

The `CoordinateMinor` action punctures the last two coordinates before the two branches are combined.

Excerpt of the final stabilizer matrix.

$$H = \left[\begin{array}{cc|c} 10000000 \dots 00000000 & 00000000 \dots 00000000 & \\ 00000000 \dots 00000000 & 10000000 \dots 00000000 & \\ 01000000 \dots 00000000 & 00000000 \dots 00000000 & \\ \vdots & \vdots & \\ 00000000 \dots 01000011 & 00000000 \dots 00000000 & \\ 00000000 \dots 00000000 & 00000000 \dots 01011111 & \end{array} \right] \in \mathbb{F}_2^{240 \times 480}.$$

B Evaluator and benchmark details

We first give the evaluator subroutines, then report validation checks, method configurations, and benchmark details supporting the main experiments.

B.1 Evaluator subroutines

Algorithms 2–6 define the subroutines called by Algorithm 1.

Algorithm 2 CANONICALIZECANDIDATE: canonicalization and parameter recovery

```

1: Input submitted matrix  $H$ 
2: if  $H$  is not binary or  $\text{ncol}(H)$  is odd then
3:   return (invalid,  $\perp$ ,  $\perp$ ,  $\perp$ )
4:  $\hat{n} \leftarrow \text{ncol}(H)/2$ ; write  $H = [H_X \mid H_Z]$ 
5:  $\bar{H} \leftarrow$  nonzero rows of  $\text{RREF}_{\mathbb{F}_2}(H)$ 
6: if  $\bar{H}_X \bar{H}_Z^\top + \bar{H}_Z \bar{H}_X^\top \neq 0$  then
7:   return (invalid,  $\perp$ ,  $\perp$ ,  $\perp$ )
8:  $\hat{k} \leftarrow \hat{n} - \text{nrow}(\bar{H})$ 
9: return (valid,  $\bar{H}$ ,  $\hat{n}$ ,  $\hat{k}$ )

```

B.2 Validation of exact-distance results

An exact distance is accepted only if all validation checks pass. The value must be an integer satisfying

$$1 \leq \ell \leq d_\star \leq u \leq n.$$

Enumeration rechecks witness weight, commutation, and exclusion from the stabilizer row space. The MacWilliams calculation accepts the first $j \geq 1$ with $B_j - A_j > 0$. The resulting value is then checked against the current bounds $[\ell, u]$.

Algorithm 3 COMPUTEDISTANCEBOUNDS: bounds for stabilizer codes with $k > 0$

```
1: Input canonical  $\bar{H}$  with  $k > 0$  and remaining budget  $B$ 
2:  $(\ell, u) \leftarrow (1, n)$ 
3: if the remaining budget is insufficient for another distance computation then
4:   return  $(\ell, u)$ 
5:  $(e, w_c, B') \leftarrow \text{ENUMERATELOGICALOPERATORS}(\bar{H}, B)$  ▷ Algorithm 4
6:  $\ell \leftarrow w_c + 1$ 
7: if  $e \neq \perp$  then
8:    $w \leftarrow \text{wt}_P(e)$ 
9:   Recheck  $\langle \bar{h}_i, e \rangle_s = 0$  for every row  $\bar{h}_i, e \notin \text{rowspan}(\bar{H})$ , and  $\text{wt}_P(e) = w$ 
10:   $\rho_e \leftarrow$  successful enumeration record for  $\bar{H}$  and  $e$ 
11:  if  $\text{VALIDATEEXACTDISTANCE}(w, [\ell, u], n, \rho_e) = \text{accepted}$  then ▷ Algorithm 6
12:    return  $(w, w)$ 
13:  $(s, d_\star) \leftarrow \text{COMPUTEEXACTDISTANCE}(\bar{H}, [\ell, u], B')$  ▷ Algorithm 5
14: if  $s = \text{accepted}$  then
15:   return  $(d_\star, d_\star)$ 
16: return  $(\ell, u)$ 
```

Algorithm 4 ENUMERATELOGICALOPERATORS: exhaustive enumeration by Pauli weight

```
1: Input canonical  $\bar{H}$  and remaining budget  $B$ 
2:  $w_c \leftarrow 0$ 
3: for  $w = 1, \dots, n$  do
4:   for each  $n$ -qubit Pauli  $e$  of weight  $w$  do
5:     if the remaining budget is insufficient for the next Pauli test then
6:       return  $(\perp, w_c, B)$ 
7:     Decrement  $B$  by one
8:     if  $\langle \bar{h}_i, e \rangle_s = 0$  for every  $i$  and  $e \notin \text{rowspan}(\bar{H})$  then
9:       return  $(e, w - 1, B)$ 
10:   $w_c \leftarrow w$ 
11: return  $(\perp, w_c, B)$ 
```

The Qubitserf adapter canonicalizes the row space, converts $[H_X \mid H_Z]$ to the native $[H_Z \mid H_X]$ convention, and requires normal completion, zero exit status, a proven flag, and a value within $[\ell, u]$ for that input. A timeout, malformed result, or contradiction is rejected. In any of these cases, no exact distance is accepted. These conditions do not provide an independently checked formal certificate.

For $k = 0$, the reported quantity is the minimum nonzero stabilizer weight, $\delta(H) = \min\{\text{wt}_P(s) : s \in \text{rowspan}(H), s \neq 0\}$. A canonical nonzero generator supplies an upper-bound witness. For rank at most 20, the complete stabilizer distribution determines δ exactly at the first $j \geq 1$ with $A_j > 0$; otherwise, exhaustive enumeration by Pauli weight excludes smaller row-space members until either a stabilizer is found or the shared per-cell Pauli-test budget is exhausted. The same bounds then determine Pass, Fail, or Unresolved. The corpus contains 47 stabilizer-state targets; method outcomes are available for 46 and are reported separately from code distance.

B.3 Evaluation diagnostics

For check j , agreement is C_j/M_j , the number of consistent cases divided by the number evaluated. The exact-distance rate is the fraction of selected candidates with $\ell(H) = u(H)$. Table 6 reports checks on instances with known results. Table 7 reports exact distance determination for one selected candidate with $k > 0$ and the requested (n, k) in each target–method pair. Neither is a primary benchmark metric.

Algorithm 5 COMPUTEEXACTDISTANCE: exact-distance computation

```
1: Input canonical  $\bar{H}$ , interval  $[\ell, u]$ , remaining budget  $B$ 
2: if the remaining budget is insufficient for the applicable exact-distance routine then
3:   return (rejected,  $\perp$ )
4: if  $r = \text{rank}(\bar{H}) \leq 20$  then
5:   Enumerate all  $2^r$  stabilizers and form their weight counts  $A_0, \dots, A_n$ 
6:    $B_j \leftarrow 2^{-r} \sum_{i=0}^n A_i K_j^{(4)}(i)$  for  $j = 0, \dots, n$   $\triangleright K_j^{(4)}$ : quaternary Krawtchouk polynomial
7:    $d_\star \leftarrow \min\{j \geq 1 : B_j - A_j > 0\}$ 
8:    $\rho \leftarrow$  successful MacWilliams computation for  $\bar{H}$ 
9: else
10:  Convert  $\bar{H}$  from  $[H_X \mid H_Z]$  to Qubitserf's  $[H_Z \mid H_X]$  convention
11:  Run single-threaded Brouwer–Zimmermann search with the specified time limit
12:  if Qubitserf does not terminate normally, returns nonzero status, or sets proven to false then
13:    return (rejected,  $\perp$ )
14:   $d_\star \leftarrow$  integer distance returned by Qubitserf
15:   $\rho \leftarrow$  successful Qubitserf computation for  $\bar{H}$ 
16:  $s \leftarrow \text{VALIDATEEXACTDISTANCE}(d_\star, [\ell, u], n, \rho)$   $\triangleright$  Algorithm 6
17: return ( $s$ ,  $d_\star$ )
```

Algorithm 6 VALIDATEEXACTDISTANCE: validation of an exact-distance result

```
1: Input value  $d_\star$ , bounds  $[\ell, u]$ , length  $n$ , computation record  $\rho$ 
2: if  $\rho$  does not record successful completion for this canonical input then
3:   return rejected
4: if  $d_\star \notin \mathbb{Z}$  or not  $1 \leq \ell \leq d_\star \leq u \leq n$  then
5:   return rejected
6: return accepted
```

Table 6: Agreement on validation cases.

Check	Agreement
Structural validation	49,576/49,576
Replay of mapped actions	29,631/29,631
Replay of ConstructionX	192/192
Validity of distance intervals	18,155/18,155
Exact-distance agreement for reference codes	79/79

Table 7: Exact-distance determination.

Method	Exact/selected
Random QT search	2,020/2,356 (85.7%)
Matched random QT search	2,019/2,356 (85.7%)
Genetic QT search	1,849/2,354 (78.5%)
Uniform action sampling	1,847/3,581 (51.6%)
MCTS	1,787/3,581 (49.9%)
Neural policy	1,966/3,581 (54.9%)

B.4 Method configurations

Construction sequences used at most four steps, four initial states, 16 candidates per action type, and 64 candidates per search state. Matched random algebraic search and genetic search invoked the evaluator after attempts 16, 32, 48, and 64. The neural policy used two width-32 tanh heads trained for 20 epochs with batch size 128, learning rate 10^{-2} , $L_2 = 10^{-5}$, and gradient clipping at 5; the frozen policy ranked greedily without online updates.

The four evaluator calls for each target–method pair shared the 10^6 -Pauli-test allowance. Candidates of rank at most 20 used the MacWilliams calculation; otherwise, single-threaded Qubitserf 0.1.0 ran for at most 30 seconds per external invocation. Algebraic operations used GAP 4.14 [34] and GUAVA 3.21 [2]. Runs used three pools of 298 CPU workers, with one CPU assigned to each worker; numerical libraries and Qubitserf were single-threaded.

B.5 Executable contract for the supplementary experiments

The policy-free one-step control and paired depth ablation used a frozen contract that separates search from verification,

$$B_{\text{search}} = (N_{\text{action}}, N_{\text{expand}}, N_{\text{sim}}, D_{\text{max}}) = (64, 64, \{0, 256\}, \{1, 4\}),$$

where only MCTS used 256 simulations, and

$$B_{\text{verify}} = (N_{\text{terminal}}, N_{\text{Pauli}}, N_{\text{backend}}, t_{\text{backend}}) = (4, 10^6, 4, 30 \text{ s}).$$

The earlier six-method artifact retains its original schema and was not retrospectively rewritten.

Every action sent to the executor consumed an action attempt, including an invalid action. Duplicate proposals removed before execution consumed no action attempt; every submission to the terminal evaluator consumed one terminal call. Structural-invalid, mismatched, and duplicate-row-space submissions consumed a terminal call but no Pauli test or backend call; duplicate row spaces reused the within-cell cache. The 10^6 Pauli tests and four backend calls were per cell and shared across its four terminal calls, while the 30-second deadline applied to each external backend invocation. Stabilizer states with $k = 0$ used the separate predicate described above and were excluded from the logical-code population.

The contract records three result layers. An action result states whether a legal child state was produced; a candidate result is structural-invalid, mismatched, or a distance interval; and the completed cell outcome is Pass, Fail, Unresolved, or No match. No match is assigned only after all submissions for a target–method–seed cell have ended. Contract validation replayed 24,006 one-step and 563,540 depth-ablation actions with their declared input and output types and found no replay failures. Tests of all return-branch budget deltas and of interval monotonicity on an exactly solvable $[[4, 2, 2]]$ code also passed.

B.6 Independent threshold check

An independent SAT encoding resolved 1,036 of 1,057 distinct combinations of a stabilizer row space and a distance threshold; all SAT decisions agreed with Qubitserf, and 21 SAT runs timed out. The 1,027 SAT witnesses were rechecked against Eq. (2). Nine UNSAT outcomes had no retained proof trace, so this check corroborates the binary threshold decisions, not the reported exact distances.

B.7 Calibration using hidden reference codes

We evaluated each of the 3,629 hidden reference matrices once under the benchmark’s enumeration and Qubitserf limits, without using construction sequences or catalog distance bounds. All matrices satisfied the structural and (n, k) checks; none was classified as Fail or contradicted its catalog interval. Table 3 reports which known reference codes could be classified as Pass from their matrices alone. Its rows for $k > 0$ are disjoint strata; the stabilizer-state row uses the criterion defined above. This calibration does not measure search-method performance.

For the 3,582 reference codes with $k > 0$, enumeration, the MacWilliams calculation, and Qubitserf increased the number classified as Pass from 472 to 1,475 and 1,503.

Every target classified as Pass by a search method was among the 1,502 evaluated targets whose reference matrix was also classified as Pass. Table 8 reports method outcomes on these targets; it does not replace the unconditional outcome rates.

Table 8: Method outcomes for the 1,502 evaluated targets with $k > 0$ whose reference matrices were classified as Pass.

Method	Pass	Fail	Unresolved	No match	Pass rate
Random QT search	112	1,202	2	186	7.46%
Matched random QT search	112	1,202	2	186	7.46%
Genetic QT search	131	1,182	2	187	8.72%
Uniform action sampling	351	1,151	0	0	23.37%
MCTS	411	1,088	3	0	27.36%
Neural policy	380	1,115	7	0	25.30%